

COLLEGE CODERS

React JS Lesson 16 – Loading & Error Handling (Premium Notes)

1) Why Loading State is needed

- API takes time to return data.
- User needs feedback while waiting.

Example:

```
const [loading,setLoading] = useState(true);
```

Explanation:

- loading starts as true before data arrives.

2) Why Error Handling is important

- API may fail because of network/server issues.
- Without error handling UI becomes confusing.

Example:

```
const [error,setError] = useState(null);
```

Explanation:

- error stores failure message.

3) Using useState for Loading & Error

- Separate state handles loading and error.

Example:

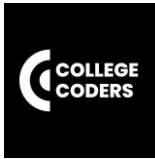
```
const [loading,setLoading] = useState(true);  
const [error,setError] = useState(null);
```

Explanation:

- Both states control UI separately.

4) Setting loading before API call

- Before fetch starts, loading should be true.



COLLEGE CODERS

Example:

```
setLoading(true);
```

Explanation:

- Shows loading UI immediately.

5) Handling API success response

- When data comes, loading stops.

Example:

```
setUsers(data);  
setLoading(false);
```

Explanation:

- Data stored and loading removed.

6) Handling API failure (catch)

- catch handles failed request.

Example:

```
.catch(()=>{  
  setError("Failed to fetch data");  
  setLoading(false);  
})
```

Explanation:

- Error UI appears.

7) Conditional Rendering in React

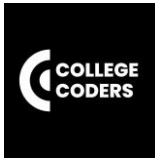
- Different UI shown for loading, error, and success.

Example:

```
if(loading) return <h2>Loading...</h2>;
```

Explanation:

- Condition decides UI.



COLLEGE CODERS

8) Loading UI, Error UI & Data UI

- Three UI states improve user experience.

Example:

```
if(error) return <h2>{error}</h2>;
```

Explanation:

- Error shown before normal data.

9) Real-world API handling flow

- Loading → Success/Error → Render final UI.

Example:

Start → Fetch → Success/Fail → Render

Explanation:

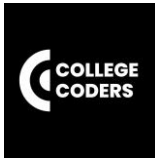
- This is standard production flow.

Mini Project – API with Loading and Error

```
const [users,setUsers] = useState([]);
const [loading,setLoading] = useState(true);
const [error,setError] = useState(null);

useEffect(()=>{
  fetch('https://jsonplaceholder.typicode.com/users')
  .then(res=>res.json())
  .then(data=>{
    setUsers(data);
    setLoading(false);
  })
  .catch(()=>{
    setError("Failed to fetch data");
    setLoading(false);
  });
},[]);
```

- Loading starts first.
- Success stores users.
- Failure stores error.



COLLEGE CODERS

- Conditional UI shows final result.